

CS 3510 Algorithms: Complexity and NP-Completeness

Aaron Hillegass
Georgia Tech

Thus far: Problems were given, we found solutions.

Today: Working with problems themselves.

Goal: To be able to say “There is no way I can find an efficient algorithm to solve this.”

Problem: We have defined lower-bounds for only a tiny number of problems.

We have a not-very-satisfying workaround

Example: Comparison Sorting

Problem: Given a list of numbers, put them in ascending order.

Solutions: Quicksort, Mergesort, Heapsort

Upper-bound? Quicksort gives us $O(n \log n)$

Lower-bound? Proof gives us $\Omega(n \log n)$

No one has ever found a polynomial-time solution to any NP-complete problem.

If we found a polynomial-time solution to any NP-complete problem, we could solve all NP-complete problems in polynomial time.

We believe, but can't prove, they are $\Omega(2^{f(n)})$ for some f .

Examples: Knapsack problem with non-integer weights, graph coloring, Traveling Salesman Problem, Boolean Satisfiability Problem, Vertex Cover Problem

Working with NP-Complete Problems

The game today is “Show that your problem is NP-Complete”. What then?

- Give up.
- Put up with an approximate solution.
- Use an SMT solver and hope your cases are easy ones.
- Do only small problems, use a lot of hardware, get your constants down.

The Worst

From “Computers and Intractability: A Guide to the Theory of NP-Completeness” (1979) by Michael Garey and David S. Johnson



“I can’t find an efficient algorithm, I guess I’m just too dumb.”

The Dream



“I can’t find an efficient algorithm, because no such algorithm is possible!”



“I can’t find an efficient algorithm, but neither can all these famous people.”

Decision Problems and P

9/21

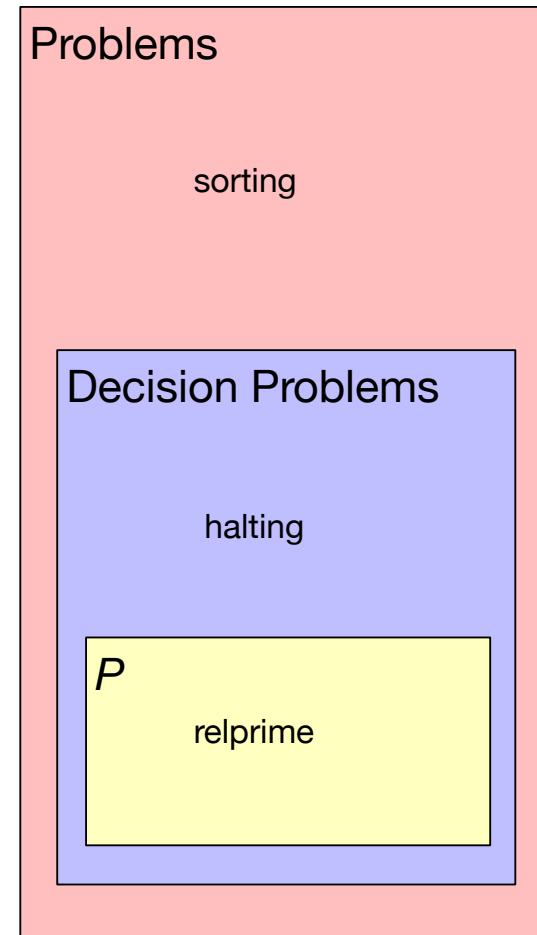
Decision problems return True or False.

Are a and b relatively prime?

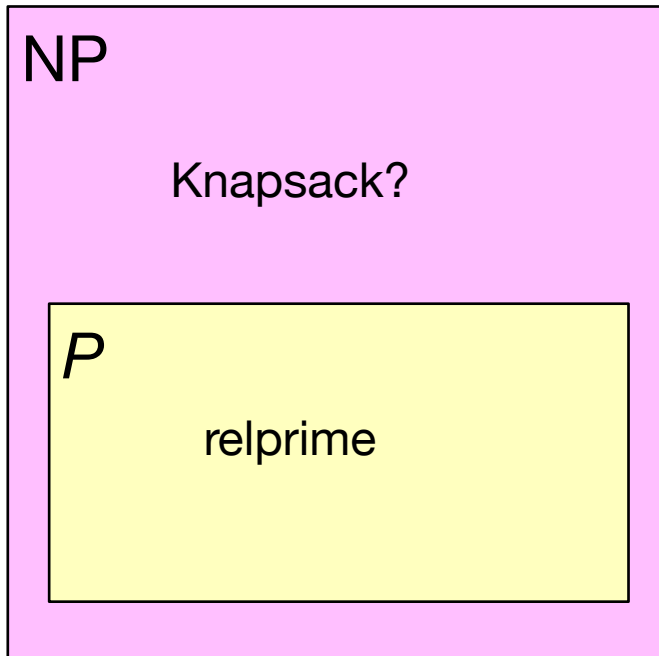
Let us work in set theory: $\{ \langle a, b \rangle \mid \gcd(a, b) = 1 \}$

Knapsack becomes a decision problem: Capacity is W , items have weights $w_1 \dots w_n$, and values $v_1 \dots v_n$. Is there a subset of items whose total weight is less than or equal to W and whose total value is greater than or equal to V ?

P is the set of all decision problems that can be solved in polynomial time.



Verifiable and NP



For the decision version of Knapsack, if the answer is True, how could you verify it?

We say the data that lets you verify is a “witness”. Usually the witness is just the solution of the non-decision form of the problem.

NP is the set of all decision problems can be verified in polynomial time.

$$P \subset NP$$

Your boss: “I don’t need an array sorted. But I have an array and I want to know the indices of where each element would be if I sorted it.”

Can you find an algorithm faster than $O(n \log n)$?

A reduction is an algorithm that transforms one problem into another.

In this setting we will be interested in polynomial time reductions of decision problems.

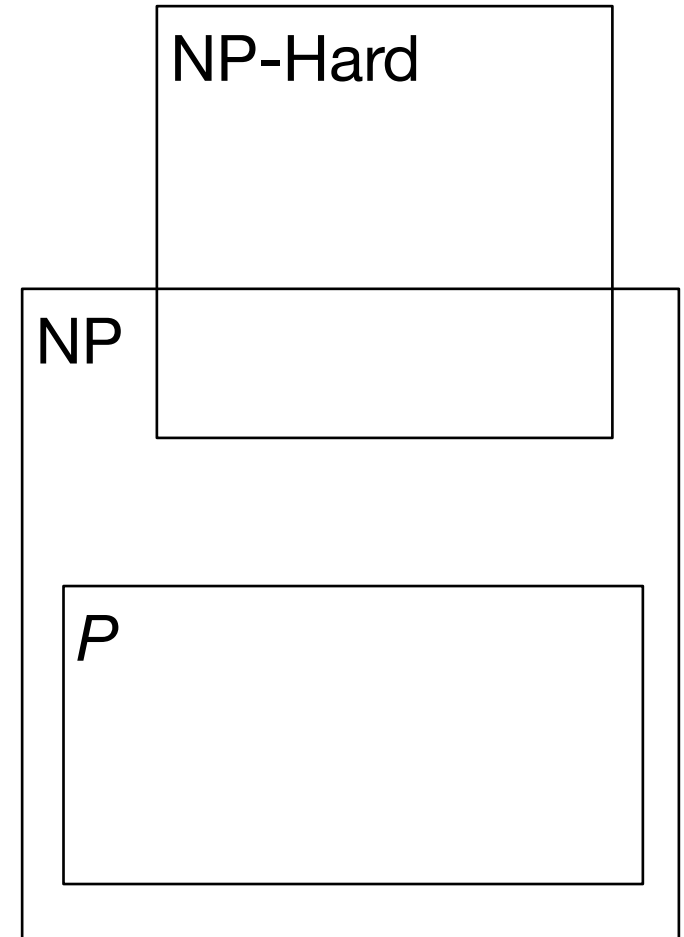
If there exists a polynomial-time reduction of problem A to problem B , we will say " $A \leq_p B$ ".

If we know A is a hard problem and $A \leq_p B$, then B is also hard.

NP-hard is a class of problems.

A problem H is NP-hard if every problem in NP can be reduced to H in polynomial time.

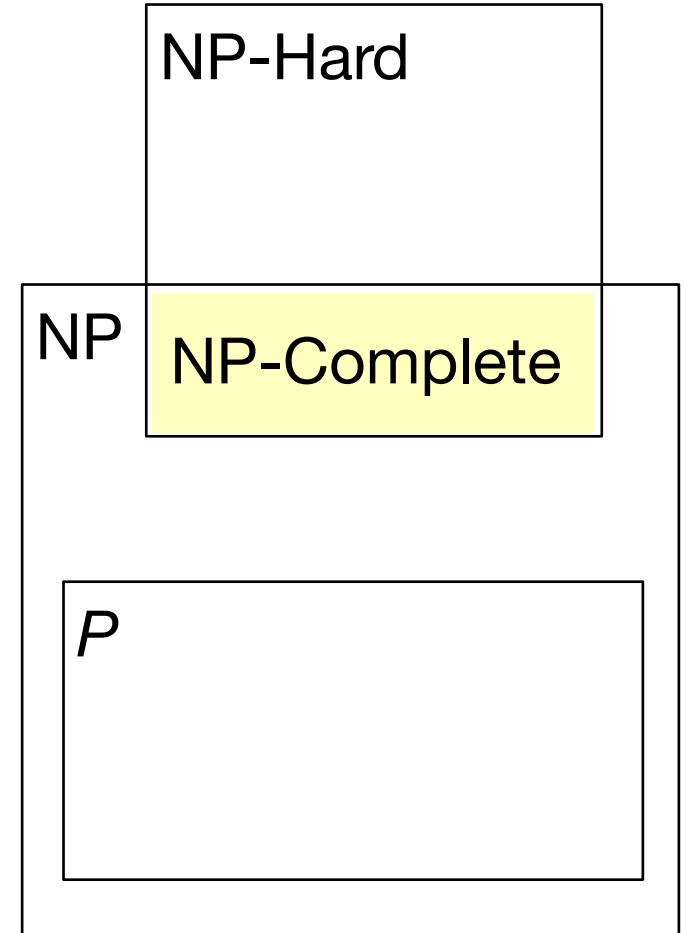
We know: $P \neq$ NP-hard



H is NP-Complete if it is in the intersection of NP and NP-Hard:

- A decision problem whose witness can be confirmed in polynomial time
- Any problem in NP can be reduced (in polynomial time) to H

The Cook-Levin Theorem gave us the first: Boolean Satisfiability is NP-Complete.



- Boolean variables: $x_1, x_2, \dots, x_n$
- Literals: $x_1, \neg x_1, x_2, \neg x_2, \dots, x_n, \neg x_n$
- Clause: Literals or-ed ($x_1 \vee \neg x_2 \vee \neg x_3 \vee x_5$)
- CNF Expression: Clauses and-ed ($(x_1 \vee \neg x_2) \wedge (\neg x_3 \vee x_5)$)

Given a CNF expression, is there an assignment that satisfies it?

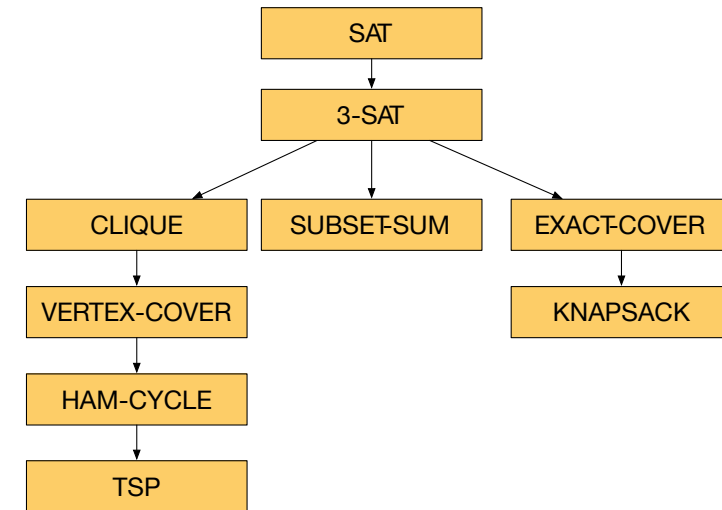
Cook-Levin Theorem proves that any problem in NP can be reduced to Boolean Satisfiability in polynomial time.

Now we have one problem in NP! How do we get others?

Using reductions!

Imagine you have a problem H in NP.

If Boolean Satisfiability can be reduced (in polynomial time) to H , then H is NP-Complete.



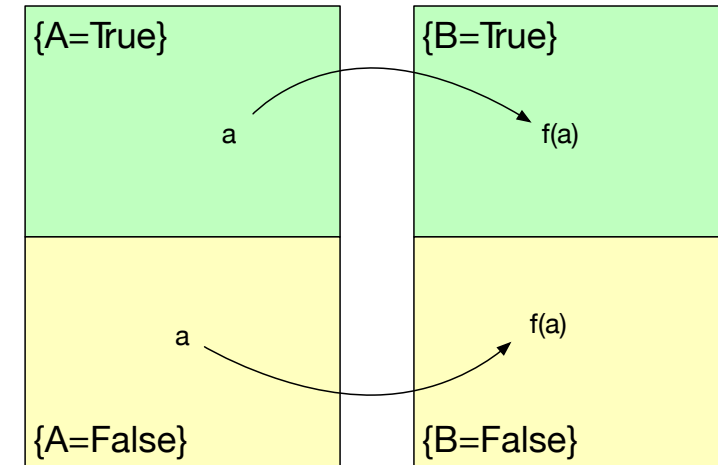
These are decision problems.

Let reduction from problem A to problem B be f

Let a be an instance of the problem A .

Then $f(a)$ is an instance of the problem B .

Then a is True in A if and only if $f(a)$ is True in B .



3-Sat: a special case of the Boolean Satisfiability problem.

Clauses can only contain three literals: $(x_1 \vee \neg x_2 \vee \neg x_3) \wedge (\neg x_1 \vee x_4 \vee \neg x_5)$

Can we reduce Boolean Satisfiability to 3-Sat?

If you have a long clause with n literals: $(r_1 \vee \dots \vee r_n)$

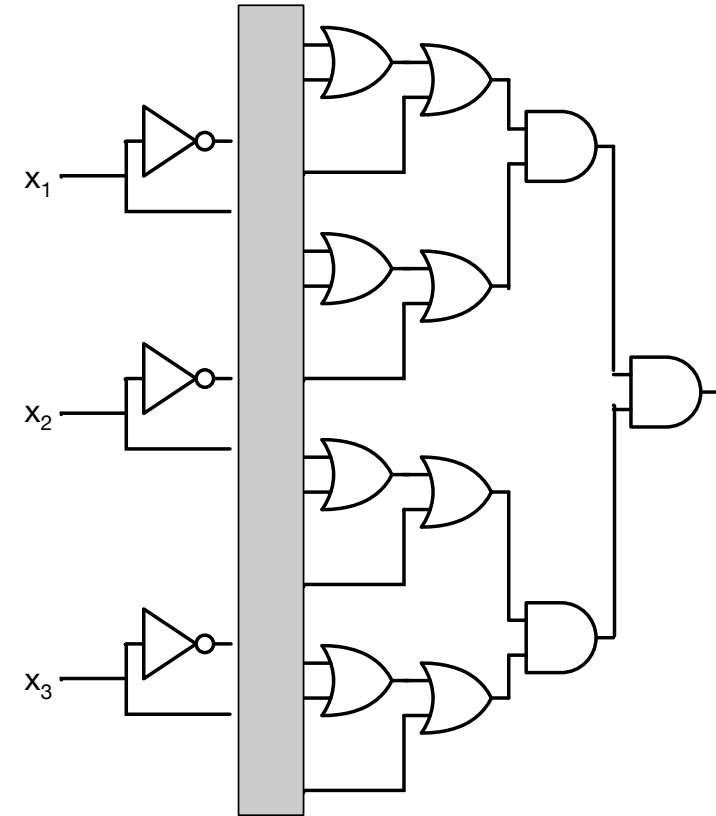
Create a new variable z and break the long clause into two:

$$(r_1 \vee \dots \vee r_{n-2} \vee z) \wedge (\neg z \vee r_{n-1} \vee r_n)$$

How long is it now?

Given a boolean circuit with n inputs and a single output, is there an input that makes the output True?

Can we reduce 3-SAT to Circuit-SAT?



Questions?

Slides by Aaron Hillegass

Based on lectures by Abraham Ladha